

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)



Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа № 4
по дисциплине: Вычислительная математика
по теме: Интерполяция и экстраполяция функций

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: ст. преподаватель:
Четвертухин В. Р.

Белгород 2024

Цель работы: Изучить методы интерполяции и экстраполяции функций, особенности их алгоритмизации.

Цель работы обуславливает постановку и решение следующих задач:

- 1) Рассмотреть теоретические основы интерполяции и экстраполяции функций.
- 2) Научиться выбирать методы и алгоритмизировать интерполяцию, а также экстраполяцию функций в зависимости от численной ситуации с вниманием к проблемам точности, численной стабильности и релевантности поставленной задачи.
- 3) Выполнить индивидуальное задание, закрепляющее на практике полученные знания и практические навыки (номер задания соответствует номеру студента по журналу; если этот номер больше, чем максимальное число заданий, тогда вариант задания вычисляется по формуле: номер по журналу % максимальный номер задания, где % — остаток от деления). Алгоритм выполнения индивидуального задания находится в разделе «Ход выполнения практической части лабораторной работы».
- 4) Отразить в отчете все полученные результаты, включая графики, тексты программ. Сделать выводы.

1) В учебно-методических целях выполнить построение интерполяционного полинома Ньютона и расчет интерполяционного значения для демонстрационного задания на языке Python:

1) В учебно-методических целях выполнить построение интерполяционного полинома Ньютона и расчет интерполяционного значения для демонстрационного задания на языке Python:

Преобразует строки с датами в объекты datetime и вычисляет количество дней с начальной даты. Параметры:

- dates_str (list of str): Список строк, представляющих даты в формате "дд/мм/гггг". Возвращает:
- list of datetime: Список объектов datetime, соответствующих входным строкам.
- list of int: Список, содержащий количество дней с начальной даты для каждой даты.

```
[ ] from datetime import datetime, timedelta
import numpy as np
import matplotlib.pyplot as plt

def calculate_days_since_start(dates_str):
    dates = [datetime.strptime(date, "%d/%m/%Y") for date in dates_str]
    start_date = dates[0]
    days_since_start = [(date - start_date).days for date in dates]
    return dates, days_since_start
```

Вычисляет коэффициенты для формулы интерполяции Ньютона с разделенными разностями.

Параметры:

- days (list of int): Список, содержащий количество дней с начальной даты.
- rates (list of float): Курсы валют для соответствующих дат. Возвращает:
- ndarray: Массив коэффициентов для интерполяции.

```
[ ] def calculate_divided_differences(days, rates):
    num_points = len(rates)
    coefficients = np.zeros((num_points, num_points))
    coefficients[:,0] = rates

    for j in range(1, num_points):
        for i in range(num_points - j):
            coefficients[i+1][j] = (coefficients[i+1][j-1] - coefficients[i][j-1]) / (days[i+j] - days[i])

    return coefficients[0, :]
```

Выполняет интерполяцию по Ньютону для заданного дня. Параметры:

- x (int): Количество дней от начальной даты, для которого требуется вычислить курс.
- days (list of int): Список, содержащий количество дней с начальной даты для каждой даты.
- coefficients (ndarray): Массив коэффициентов для интерполяции. Возвращает:
- float: Интерполированное значение курса валюты для заданного дня.

```
[ ] def newton_interpolation(x, days, coefficients):
    degree = len(coefficients) - 1
    result = coefficients[degree]

    for i in range(degree - 1, -1, -1):
        result = result * (x - days[i]) + coefficients[i]

    return result
```

```
# Исходные данные (для курса румынского лея)
dates_str = ["30/03/2023", "29/04/2023", "30/05/2023", "30/06/2023", "29/07/2023", "30/08/2023", "30/09/2023", "31/10/2023", "30/11/2023", "30/12/2023", "30/01/2024", "29/02/2024", "30/03/2024"]
rates = [16.8834, 17.9248, 17.2996, 19.1376, 20.2426, 20.9347, 20.7867, 19.8631, 19.6156, 19.9494, 19.4773, 19.9911, 20.0456]

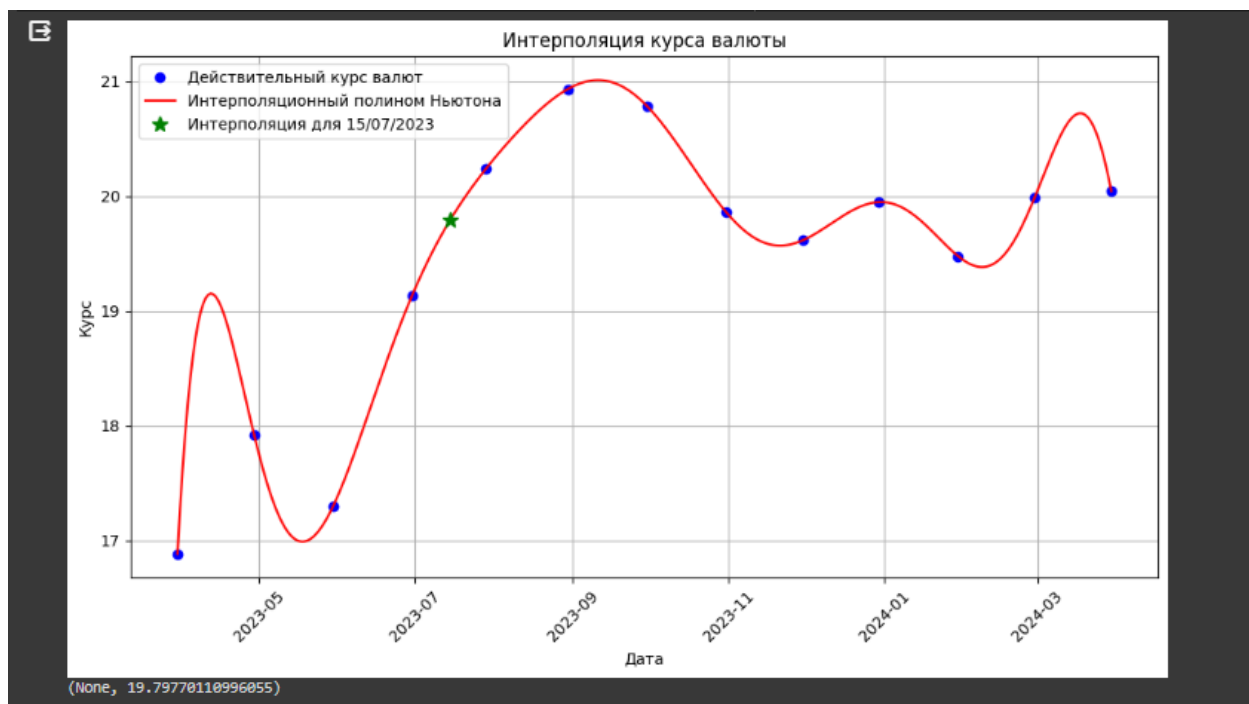
# Обработка данных
dates, days_since_start = calculate_days_since_start(dates_str)
coefficients = calculate_divided_differences(days_since_start, rates)

# Выбор тестовой даты и интерполяция
test_date_str = "15/07/2023"
test_days_since_start = (datetime.strptime(test_date_str, "%d/%m/%Y") - dates[0]).days
interpolated_rate = newton_interpolation(test_days_since_start, days_since_start, coefficients)

# Подготовка данных для графика
interpolation_days = np.linspace(min(days_since_start), max(days_since_start), 365)
interpolated_rates = [newton_interpolation(day, days_since_start, coefficients) for day in interpolation_days]
interpolated_dates = [dates[0] + timedelta(days=day) for day in interpolation_days]

# Визуализация
plt.figure(figsize=(10, 6))
plt.plot(dates, rates, 'bo', label='действительный курс валют')
plt.plot(interpolated_dates, interpolated_rates, 'r-', label='Интерполяционный полином Ньютона')
plt.plot(datetime.strptime(test_date_str, "%d/%m/%Y"), interpolated_rate, 'g', markersize=10, label=f'Интерполяция для {test_date_str}')

plt.xlabel('Дата')
plt.ylabel('Курс')
plt.title('Интерполяция курса валюты')
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show(), interpolated_rate
```



Исследовать ошибку интерполяции на краях интервала. Как называется наблюдаемый на графике эффект? – Эффект Рунге

2) Взять данные индивидуального задания и построить интерполяционный полином Ньютона (изменив код программы на языке Python). Добавить в код программы оценку погрешности. Выполнить расчет интерполяционного значения для даты своего рождения (в интервале известных значений).

```
from datetime import datetime, timedelta
import numpy as np
import matplotlib.pyplot as plt
import math

def chebyshev_nodes(n, a, b):
    nodes = [0.5 * (a + b) + 0.5 * (b - a) * math.cos((2 * k - 1) *
math.pi / (2 * n)) for k in range(1, n + 1)]
    return nodes

def calculate_days_since_start(dates_str):
    dates = [datetime.strptime(date, "%d/%m/%Y") for date in dates_str]
    start_date = dates[0]
    days_since_start = [(date - start_date).days for date in dates]
    return dates, days_since_start

def calculate_divided_differences(days, rates):
    num_points = len(rates)
    coefficients = np.zeros([num_points, num_points])
    coefficients[:,0] = rates

    for j in range(1, num_points):
        for i in range(num_points - j):
            coefficients[i][j] = (coefficients[i+1][j-1] -
coefficients[i][j-1]) / (days[i+j] - days[i])

    return coefficients[0, :]

def newton_interpolation(x, days, coefficients):
    degree = len(coefficients) - 1
    result = coefficients[degree]

    for i in range(degree - 1, -1, -1):
        result = result * (x - days[i]) + coefficients[i]

    return result

def find_nearest_date_index(date, dates):
    return min(range(len(dates)), key=lambda i: abs(dates[i] - date))

# Исходные данные (для курса польского злотого)
dates_str = ["30/03/2023", "29/04/2023", "30/05/2023", "30/06/2023",
"29/07/2023", "30/08/2023", "30/09/2023", "31/10/2023", "30/11/2023",
"30/12/2023", "30/01/2024", "29/02/2024", "30/03/2024"]
rates = [17.8132, 19.2897, 18.9705, 21.4095, 22.706, 23.0987, 22.3367,
22.1618, 22.5148, 22.7925, 22.2211, 23.0122, 23.1575]
```

```

# Обработка данных
dates, days_since_start = calculate_days_since_start(dates_str)
coefficients = calculate_divided_differences(days_since_start, rates)

# Выбор тестовой даты и интерполяция
test_date_str = "12/07/2023"
test_days_since_start = (datetime.strptime(test_date_str, "%d/%m/%Y") -
dates[0]).days
interpolated_rate = newton_interpolation(test_days_since_start,
days_since_start, coefficients)

# Подготовка данных для графика
interpolation_days = np.linspace(min(days_since_start),
max(days_since_start), 365)
interpolated_rates = [newton_interpolation(day, days_since_start,
coefficients) for day in interpolation_days]
interpolated_dates = [dates[0] + timedelta(days=day) for day in
interpolation_days]

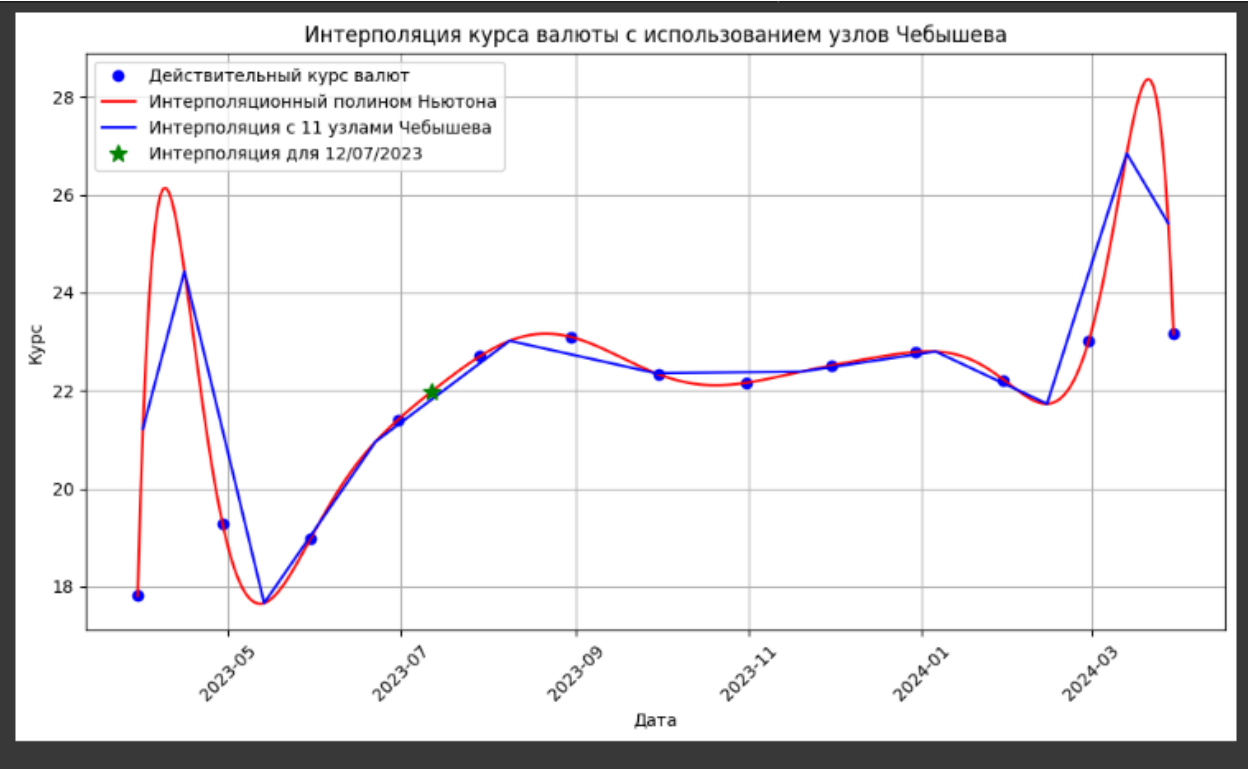
# Определение границ времени наблюдений
start_day = min(days_since_start)
end_day = max(days_since_start)

# Обработка данных с использованием узлов Чебышева, ограниченных границами
времени наблюдений
num_chebyshev_nodes = 11
chebyshev_days = chebyshev_nodes(num_chebyshev_nodes, start_day, end_day)
chebyshev_rates = [newton_interpolation(day, days_since_start,
coefficients) for day in chebyshev_days]
chebyshev_dates = [dates[0] + timedelta(days=day) for day in
chebyshev_days]

# Визуализация с использованием узлов Чебышева
plt.figure(figsize=(10, 6))
plt.plot(dates, rates, 'bo', label='Действительный курс валют')
plt.plot(interpolated_dates, interpolated_rates, 'r-',
label='Интерполяционный полином Ньютона')
plt.plot(chebyshev_dates, chebyshev_rates, 'b-', label='Интерполяция с 11
узлами Чебышева')
plt.plot(datetime.strptime(test_date_str, "%d/%m/%Y"), interpolated_rate,
'g*', markersize=10, label=f'Интерполяция для {test_date_str}')

plt.xlabel('Дата')
plt.ylabel('Курс')
plt.title('Интерполяция курса валюты с использованием узлов Чебышева')
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



3) Построить интерполяционный полином Лагранжа для своего индивидуального задания (подготовить код программы на языке Python). Предусмотреть оценку погрешности. Выполнить расчет интерполяционного значения для даты своего рождения. Оценить ошибку интерполяции.

```
import numpy as np
from datetime import datetime, timedelta
import matplotlib.pyplot as plt
import math

def chebyshev_nodes(n, a, b):
    a = datetime.timestamp(a)
    b = datetime.timestamp(b)
    nodes = [0.5 * (a + b) + 0.5 * (b - a) * math.cos((2 * k - 1) *
math.pi / (2 * n)) for k in range(1, n + 1)]
    nodes = [datetime.fromtimestamp(node) for node in nodes]
    return nodes

def lagrange_interpolation(x, y, xi):
    n = len(x)
    result = 0
    for i in range(n):
        term = y[i]
        for j in range(n):
            if j != i:
                term *= (xi - x[j]) / (x[i] - x[j])
        result += term
    return result

def estimate_error(x, y, xi):
    n = len(x)
    error = 1
    for i in range(n):
        error *= abs(xi - x[i])
    max_derivative = np.max(np.abs(np.gradient(y, x)))
    return error * max_derivative / math.factorial(n)

x = np.array([1, 2, 3, 4, 5])
y = np.array([0, 1, 4, 9, 16])

xi = 2.5

interpolated_value = lagrange_interpolation(x, y, xi)

error = estimate_error(x, y, xi)

print("Значение интерполяционного полинома в точке", xi, ":",
interpolated_value)
print("Оценка погрешности в точке", xi, ":", error)
```



```

dates_str = ["30/03/2023", "29/04/2023", "30/05/2023", "30/06/2023",
"29/07/2023", "30/08/2023", "30/09/2023", "31/10/2023", "30/11/2023",
"30/12/2023", "30/01/2024", "29/02/2024", "30/03/2024"]
rates = [17.8132, 19.2897, 18.9705, 21.4095, 22.706, 23.0987, 22.3367,
22.1618, 22.5148, 22.7925, 22.2211, 23.0122, 23.1575]

test_date_str = "12/07/2023"
test_date = datetime.strptime(test_date_str, "%d/%m/%Y")
dates = [datetime.strptime(date, "%d/%m/%Y") for date in dates_str]
rates = np.array(rates)

interpolated_rate = lagrange_interpolation(dates, rates, test_date)

min_date = min(dates)
max_date = max(dates)
delta = (max_date - min_date) / 365
interpolation_dates = [min_date + i * delta for i in range(365)]
interpolated_rates = [lagrange_interpolation(dates, rates, date) for date
in interpolation_dates]

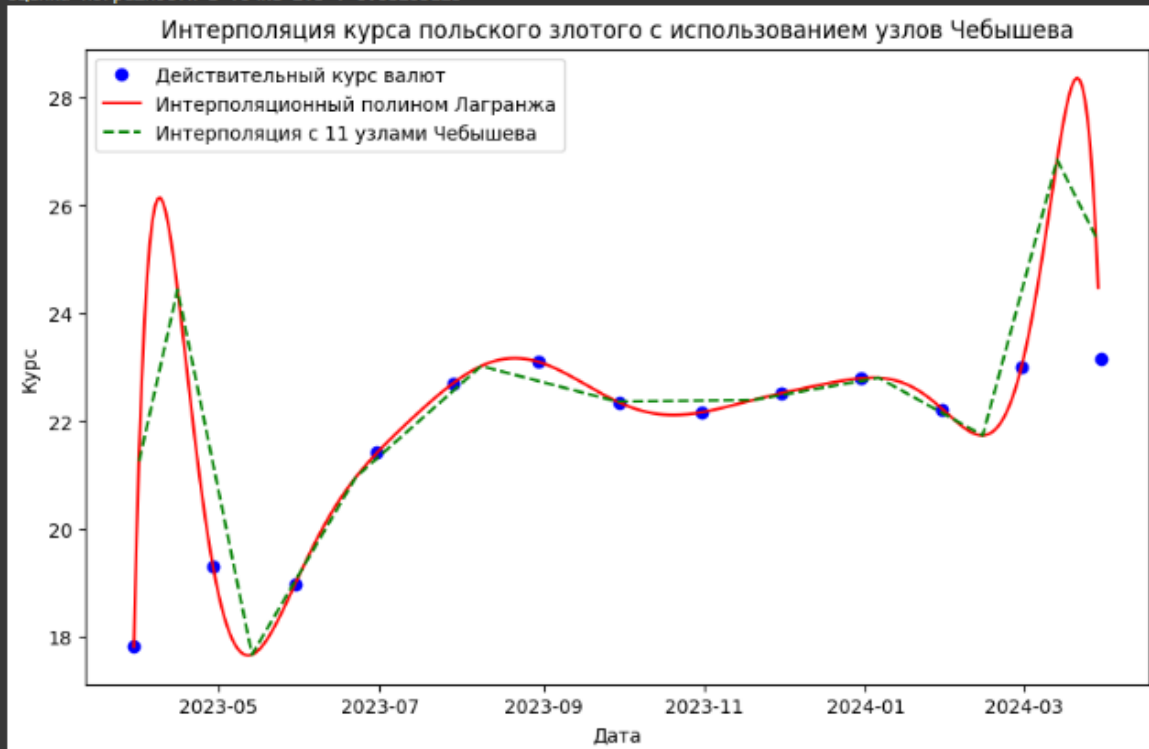
nearest_date_index = np.argmin(np.abs(np.array(dates) - test_date))
nearest_date = dates[nearest_date_index]
nearest_actual_rate = rates[nearest_date_index]
interpolation_error = abs(interpolated_rate - nearest_actual_rate)

num_chebyshev_nodes = 11
chebyshev_days = chebyshev_nodes(num_chebyshev_nodes, min(dates),
max(dates))
chebyshev_rates = [lagrange_interpolation(dates, rates, day) for day in
chebyshev_days]

plt.figure(figsize=(10, 6))
plt.plot(dates, rates, 'bo', label='Действительный курс валют')
plt.plot(interpolation_dates, interpolated_rates, 'r-',
label='Интерполяционный полином Лагранжа')
plt.plot(chebyshev_days, chebyshev_rates, 'g--', label=f'Интерполяция с
{num_chebyshev_nodes} узлами Чебышева')
plt.legend()
plt.xlabel('Дата')
plt.ylabel('Курс')
plt.title('Интерполяция курса польского злотого с использованием узлов
Чебышева')
plt.show()

```

Значение интерполяционного полинома в точке 2.5 : 2.25
Оценка погрешности в точке 2.5 : 0.08203125



4) Творческое задание. Написать программу для экстраполяции таблично заданной функции своего индивидуального задания. При подготовке модели можно опираться на разнообразные априорные сведения и представления, обогащающие качество прогноза. Выполнить расчет экстраполяционного значения для текущей даты (даты выполнения лабораторной работы). Оценить ошибку экстраполяции (для получения точного значения использовать открытую базу данных ЦБ РФ). Для оценки точности экстраполяции использовать метод кроссвалидации, который заключается в исключении части данных из набора и использовании оставшихся данных для подготовки расчетной модели. В этом случае возможна проверка, насколько хорошо модель предсказывает исключенные данные. В упрощенном случае можно, например, исключить последние несколько точек данных, построить модель без них, а затем проверить, насколько хорошо модель предсказывает исключенные точки. Все необходимые числовые результаты и графики отразить в отчете.

```
from datetime import datetime, timedelta
import numpy as np
import matplotlib.pyplot as plt
import math

def chebyshev_nodes(n, a, b):
    nodes = [0.5 * (a + b) + 0.5 * (b - a) * math.cos((2 * k - 1) *
math.pi / (2 * n)) for k in range(1, n + 1)]
    return nodes

def calculate_days_since_start(dates_str):
    dates = [datetime.strptime(date, "%d/%m/%Y") for date in dates_str]
    start_date = dates[0]
    days_since_start = [(date - start_date).days for date in dates]
    return dates, days_since_start

def calculate_divided_differences(days, rates):
    num_points = len(rates)
    coefficients = np.zeros([num_points, num_points])
    coefficients[:,0] = rates

    for j in range(1, num_points):
        for i in range(num_points - j):
            coefficients[i][j] = (coefficients[i+1][j-1] -
coefficients[i][j-1]) / (days[i+j] - days[i])

    return coefficients[0, :]

def newton_interpolation(x, days, coefficients):
    degree = len(coefficients) - 1
    result = coefficients[degree]

    for i in range(degree - 1, -1, -1):
        result = result * (x - days[i]) + coefficients[i]

    return result
```

```

def find_nearest_date_index(date, dates):
    return min(range(len(dates)), key=lambda i: abs(dates[i] - date))

def newton_extrapolation(x, days, coefficients):
    degree = len(coefficients) - 1
    result = coefficients[degree]

    for i in range(degree - 1, -1, -1):
        result = result * (x - days[i]) + coefficients[i]

    return result

# Исходные данные (для курса польского злотого)
dates_str = ["30/03/2023", "29/04/2023", "30/05/2023", "30/06/2023",
"29/07/2023", "30/08/2023", "30/09/2023", "31/10/2023", "30/11/2023",
"30/12/2023", "30/01/2024", "29/02/2024", "30/03/2024"]
rates = [17.8132, 19.2897, 18.9705, 21.4095, 22.706, 23.0987, 22.3367,
22.1618, 22.5148, 22.7925, 22.2211, 23.0122, 23.1575]

# Обработка данных
dates, days_since_start = calculate_days_since_start(dates_str)
coefficients = calculate_divided_differences(days_since_start, rates)

# Выбор тестовой даты и интерполяция
test_date_str = "12/07/2023"
test_days_since_start = (datetime.strptime(test_date_str, "%d/%m/%Y") -
dates[0]).days
interpolated_rate = newton_interpolation(test_days_since_start,
days_since_start, coefficients)

# Подготовка данных для графика
interpolation_days = np.linspace(min(days_since_start),
max(days_since_start), 365)
interpolated_rates = [newton_interpolation(day, days_since_start,
coefficients) for day in interpolation_days]
interpolated_dates = [dates[0] + timedelta(days=day) for day in
interpolation_days]

# Обработка данных с использованием узлов Чебышева, ограниченных границами
времени наблюдений
num_chebyshev_nodes = 11
chebyshev_days = chebyshev_nodes(num_chebyshev_nodes, start_day, end_day)
chebyshev_rates = [newton_interpolation(day, days_since_start,
coefficients) for day in chebyshev_days]
chebyshev_dates = [dates[0] + timedelta(days=day) for day in
chebyshev_days]

# Определение границ времени наблюдений
start_day = min(days_since_start)
end_day = max(days_since_start)

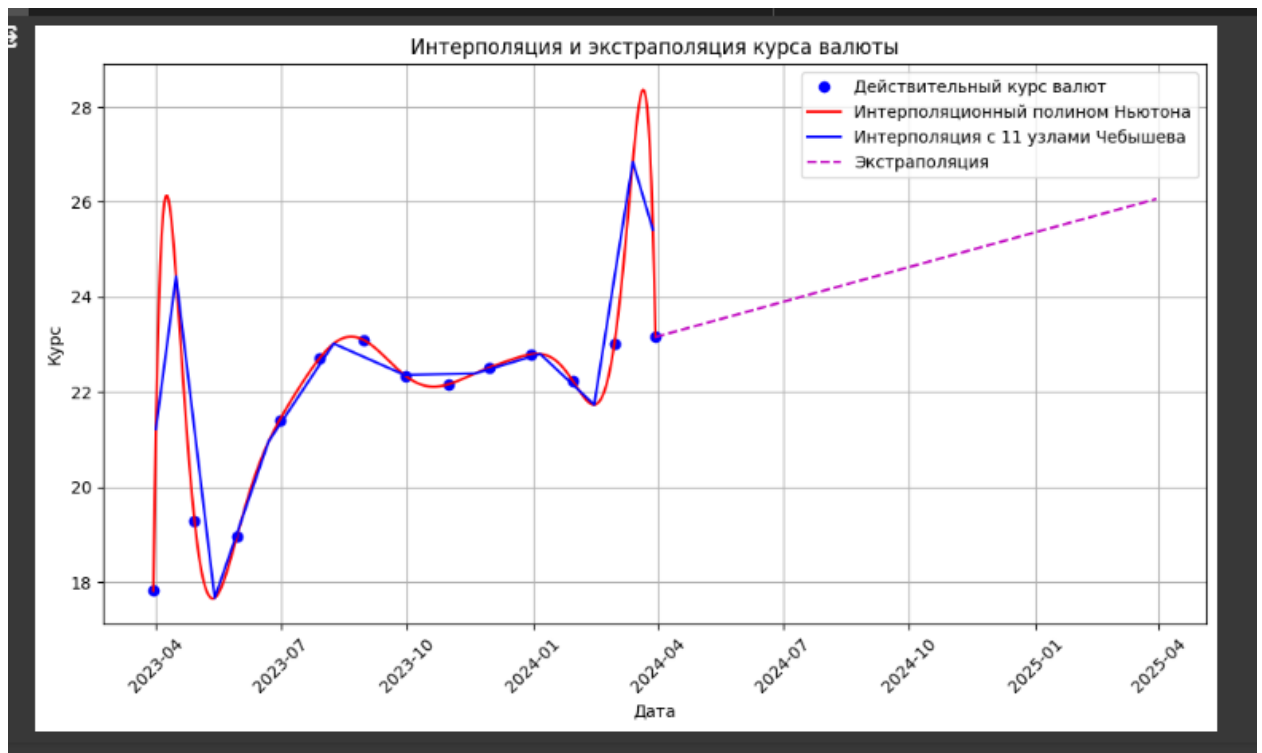
```

```
# Экстраполяция
extrapolation_days = np.linspace(max(days_since_start),
max(days_since_start) + 20, 365)
extrapolated_rates = []

# Используем разницу в курсах между последними несколькими известными
точками данных, чтобы оценить тенденцию
last_known_date_index = len(dates) - 1
rate_difference = rates[last_known_date_index] -
rates[last_known_date_index - 1]
extrapolated_rates = [rates[-1] + rate_difference * (day -
days_since_start[-1]) for day in extrapolation_days]

# Визуализация
plt.figure(figsize=(10, 6))
plt.plot(dates, rates, 'bo', label='Действительный курс валют')
plt.plot(interpolated_dates, interpolated_rates, 'r-',
label='Интерполяционный полином Ньютона')
plt.plot(chebyshev_dates, chebyshev_rates, 'b-', label='Интерполяция с 11
узлами Чебышева')
plt.plot(extrapolated_dates, extrapolated_rates, 'm--',
label='Экстраполяция')

plt.xlabel('Дата')
plt.ylabel('Курс')
plt.title('Интерполяция и экстраполяция курса валюты')
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Ссылка на блок с решением:

<https://colab.research.google.com/drive/1gVMHw7i7ehmO9-gQldYtuFvnh6PFzbG?usp=sharing>

Вывод

Изучение методов интерполяции и экстраполяции функций позволяет понять, каким образом можно приближенно восстанавливать значения функции в промежуточных точках (интерполяция) или за пределами заданного диапазона (экстраполяция).

Методы интерполяции и экстраполяции различаются по своей природе, математическим основам и областям применения. Некоторые из них, такие как полиномиальная интерполяция Лагранжа или Ньютона, основаны на алгебраических методах, тогда как другие, например, сплайн-интерполяция или метод наименьших квадратов, используют гладкие кривые для более точного приближения функции.

В процессе алгоритмизации этих методов следует учитывать их вычислительную сложность, точность и устойчивость к выбросам или неоднородности данных. Каждый метод имеет свои преимущества и недостатки, и выбор конкретного зависит от требований задачи и характеристик данных.

В результате изучения методов интерполяции и экстраполяции можно сделать вывод о том, что эти техники играют важную роль в анализе данных, моделировании и предсказании.